

SMBCI Data Engineer Technical Test: CDC Challenge

Context

Your company operates a PostgreSQL database for its main application. You want to build a simple **CDC (Change Data Capture)** pipeline to capture and process **row-level changes** (inserts, updates, deletes) into a target system (like S3, a data lake, or another database). You will be given 30 minutes to present your answers on a scheduled date.

Requirements

1. **Source Database**
 - PostgreSQL database with a table called `customers(id, name, email, updated_at)`.
2. **Change Detection**
 - Capture changes (INSERTs, UPDATEs, DELETEs) to the `customers` table.
 - Make a publisher that changes the name and email
3. **Processing Logic**
 - Continuously read changes.
 - Format each change event as JSON, including:
 - Operation type (`insert`, `update`, `delete`)
 - Full row after change (for inserts/updates)
 - Primary key info for deletes
4. **Sink (Destination)**
 - Store the JSON events into **one of the following**:
 - Local file system (one file per event)
 - Another PostgreSQL database
 - In-memory storage (list or dictionary)
5. **Extra (Optional, Bonus Points)**
 - Implement crash recovery (so the pipeline resumes after failure).
 - Use Debezium or logical decoding.
 - Calculate the latency below:
 - data lag (time difference between a change happened and when it was processed)
 - Number of change events processed per second
 - Throughput: Total data volume processed per unit time (e.g., MB/s)
 - Simple monitoring (e.g., logging the number of events processed).

🌟 Constraints

- Use any of these languages: Python, Golang, Scala, Java
- If using libraries or connectors, explain how you configured them.
- Focus on data flow correctness — not on building UIs.

🚀 Deliverables

- Source code.
- A short `README.md` including:
 - How to run it.
 - Architecture overview.
- (Optional) A short section on how to productionize it.

💻 Grading Rubric

Criteria	Weight
Correctness (CDC works)	40%
Code Quality (clear, modular)	20%
Performance (no obvious bottlenecks)	10%
Resiliency (handles crashes)	15%
Documentation (clarity)	10%
Presentation (clarity)	5%

🔥 Source Database Setup

To help you, here's how to set up a local PostgreSQL database ready for CDC:

Step 1: Start PostgreSQL with Logical Replication Enabled

```
docker run --name cdc-postgres \  
-e POSTGRES_USER=postgres \  
-e POSTGRES_PASSWORD=postgres \  

```

```
-e POSTGRES_DB=source_db \  
-p 5432:5432 \  
-d postgres:15 \  
-c wal_level=logical \  
-c max_replication_slots=10 \  
-c max_wal_senders=10
```

Step 2: Create the Customers Table

Connect to the database:

```
psql -h localhost -U postgres -d source_db
```

Create the table:

```
CREATE TABLE customers (  
    id SERIAL PRIMARY KEY,  
    name TEXT NOT NULL,  
    email TEXT NOT NULL,  
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

Insert a sample:

```
INSERT INTO customers (name, email) VALUES ('Alice', 'alice@example.com');
```

Step 3: Enable Logical Replication

Create a publication:

```
CREATE PUBLICATION mypub FOR TABLE customers;
```



Final Notes

- Think of how you would **scale** this if tables get bigger.
 - Think of how to **minimize latency**.
 - Think of **error handling and retries**.
-



Good luck, have fun!